

Assignment 3 Report: doit Shell Agent

Interactive LLM agent for bash command generation, safety, memory, sessions, and ACDL documentation

Lior Alafi 032543472, Tal Somech 315473033

June 2026

Contents

1	Requirements Coverage	3
2	Executive Summary	4
3	Repository and Installation	5
3.1	CLI layout	5
3.2	WSL/Linux installation	5
3.3	Model configuration	6
4	Architecture	6
4.1	High-level design	6
4.2	Main runtime flow	8
5	Structured Output and Prompt Design	10
5.1	Agent decision schema	10
5.2	Main prompt context	11
5.3	JSON parsing and recovery	11
6	Single Command Agent	11
7	Shell Execution and Parent-Shell Commands	12
8	Dangerous Command Detection	13
8.1	Rule-based safety	13
8.2	LLM safety fallback	13
8.3	User confirmation	14
9	Model Flexibility	15
10	ACDL Documentation	15
10.1	Two levels of ACDL	15
10.2	Prompt-level visualization	16
10.3	Scenario-level visualization	17
11	Multi-turn Interaction	18
12	Clarifications and Richer Interactions	18
13	Memory	19

14 User Awareness	20
15 Output Awareness	20
16 Multi-session and Multi-tasking	21
17 Implemented Extension: Context Summarization	22
17.1 Other extensions considered	24
18 Prompt Logging	25
19 Experiments	25
19.1 Experimental setup	25
19.2 Observed model differences	26
19.3 Latest-only behavior table	26
20 Limitations	29
20.1 Single-command formulation failures	29
20.2 Memory extraction occurs too early	29
20.3 Clarification is implemented but not robust	29
20.4 Multi-session resolution remains difficult	29
20.5 Safety classification is conservative but incomplete	29
21 Conclusion	29
A Prompt Schemas	30
A.1 Agent schema	30
A.2 Safety schema	30
A.3 Memory schema	30
B ACDL and Figure Inventory	30
C Selected Additional ACDL Visualizations	32
D Prompt Templates Used	34
D.1 Main agent prompt: DoitAgentStateful	35
D.2 Clarification prompt: DoitClarification	36
D.3 Safety prompt: DoitSafetyCheck	36
D.4 Memory extraction prompt: DoitMemoryExtraction	36
D.5 Context summary prompt: DoitContextSummary	37

1 Requirements Coverage

Table 1 maps the assignment requirements to the implementation and to the corresponding report sections. The assignment asks for a working doit CLI, shell execution with captured stdout/stderr/return code, dangerous-command handling, model flexibility, ACDL documentation, multi-turn behavior, clarifications, richer interactions, memory, user awareness, output awareness, multi-tasking, one implemented extension, logs, model comparison, and prompt/schema documentation.

Table 1: Coverage matrix for the assignment requirements.

Requirement	Implementation evidence	Where documented
CLI named doit, written in Python, invocable from any directory	Top-level executable doit; WSL/Linux install scripts update shell environment and use ~/doit.cfg.	Repository and Installation
Single shell command at a time	SingleCommandPolicy rejects top-level newlines, &&, , unescaped semicolons, command substitution, and backticks.	Single Command Agent; latest-only results table T3, T16
Command execution and output capture	Shell subprocess returns separate stdout, stderr, and returncode; parent-shell path handles cd/export.	Shell Execution and Parent-Shell Commands
Normal answers / impossible shell requests	Agent schema supports answer, conversation, and error, not only commands.	Structured Output; Richer Interactions; T7
Dangerous / modifying command detection	Rule-based safety plus optional LLM fallback; modifying commands require confirmation.	Dangerous Command Detection; T4, T5, T12, T13
Hosted and local model flexibility	LiteLLM + ~/doit.cfg; tested Gemini, Qwen, and Gemma.	Model Flexibility; Experiments
Three-model experimentation	Gemini as hosted API, Qwen as local structured/tool-oriented instruction model, Gemma as local general instruction model.	Model Flexibility; Latest-only behavior table
ACDL textual documentation and visuals	Prompt-level ACDL for each LLM call; scenario-level ACDL for full runtime traces; generated PNG visualizations.	ACDL Documentation; ACDL and Figure Inventory
Multi-turn across separate invocations	Persistent InteractionRecord history with same-session and other-session context.	Multi-turn Interaction
Clarifications	intent="clarification" with terminal prompt and second LLM call.	Clarifications and Richer Interactions; T18 limitation
Richer non-command interactions	answer intent for explanatory questions.	Richer Interactions; T7
Memory distinct from history	Separate memory store and memory-extraction prompt.	Memory; T11 limitation

Continued on next page

Requirement	Implementation evidence	Where documented
User awareness	Reads recent external shell history and includes it in the main prompt.	User Awareness; T15
Output awareness	Stores previous command output and error messages for follow-up questions.	Output Awareness; T14
Multi-tasking / multi-session	Uses DOIT_SESSION_ID and separate session streams, while allowing explicit cross-session references.	Multi-session and Multi-tasking; T17 limitation
Additional extension	Context summarization for long histories, plus two unimplemented alternatives described.	Implemented Extension: Context Summarization
Logs and failure cases	Prompt logs and interaction histories are included in the submission; latest-only table summarizes successes and failures.	Prompt Logging; Experiments
Prompt templates / schemas	JSON schemas and prompt templates are documented in the appendix.	Prompt Schemas; Prompt Templates Used

2 Executive Summary

This report describes `doit`, a Python command-line LLM agent that translates natural-language requests into bash commands, executes safe single commands, captures command output, and preserves context across separate invocations. The implementation supports local and hosted models through LiteLLM, filesystem-safety checks, multi-turn history, clarification questions, persistent memories, shell-history awareness, multi-terminal sessions, and a context summarization extension.

The final implementation is not only a command translator. It is an agent with three interacting layers:

1. **LLM decision layer:** produces structured JSON decisions such as `execute_command`, `answer`, `clarification`, or `error`.
2. **Runtime/tool layer:** validates the command, classifies safety, asks for confirmation when needed, executes bash, captures stdout, stderr, and returncode, and updates state.
3. **Context layer:** stores history, memories, session summaries, shell history, and separate terminal streams so future invocations can refer to previous work.

The strongest observed behavior was produced by Qwen3 4B Instruct, which generally followed the JSON schema and handled output-awareness well. Gemma3 4B was useful as a non-tool-specialized local baseline but produced more structured-output and semantic command errors. Gemini 2.5 Flash demonstrated hosted API support, but the Gemini run set was smaller.

3 Repository and Installation

3.1 CLI layout

The project is organized around a top-level executable named `doit`. Even though it has no file extension, it is a Python program. The main modules are:

- `doit`: CLI entry point.
- `doit_agent/agent.py`: main orchestration logic.
- `doit_agent/prompts.py`: prompt templates and schemas.
- `doit_agent/schemas.py`: Pydantic schemas for structured model outputs.
- `doit_agent/safety.py`: rule-based and LLM fallback safety classification.
- `doit_agent/command_policy.py`: single-command validation and parent-shell command detection.
- `doit_agent/shell_exec.py`: subprocess execution with captured output.
- `doit_agent/state/`: interaction history storage.
- `doit_agent/memory/`: long-term memory storage.
- `doit_agent/user_awareness/`: recent shell-history awareness.
- `acdl/prompt/`: ACDL files for actual LLM calls.
- `acdl/scenarios/`: ACDL files for full end-to-end runtime scenarios.

3.2 WSL/Linux installation

The installation scripts were updated so that the project can run both on Linux and in WSL. The installation flow is:

```
cd /path/to/project
bash install.bash
source ~/.bashrc
doit "list_files"
```

The installer creates a virtual environment, installs dependencies, copies `doit.cfg.example` to `~/doit.cfg` when needed, and adds the shell setup to `.bashrc`. A key WSL fix is that the shell wrapper invokes the Python interpreter from the project virtual environment:

```
./venv/bin/python
```

This avoids the common error where dependencies are installed inside `.venv`, but `doit` is executed with the system Python and fails with `ModuleNotFoundError: litellm`.

Another important fix is that the executable now loads the user-level config:

```
config = ConfigLoader().load()
```

instead of loading a local project `doit.cfg` file. This matches the assignment requirement that model configuration is selected using `~/doit.cfg`.

3.3 Model configuration

The configuration file controls the selected provider and model. The implementation supports local Ollama models and hosted providers through LiteLLM. A typical local configuration is:

```
[model]
provider = ollama
model = ollama/qwen3:4b-instruct
api_base = http://localhost:11434

[agent]
shell = /bin/bash
secure = true
max_clarification_rounds = 2
command_timeout = 20
```

For WSL with Ollama running on Windows, the Windows host can be detected with:

```
WIN_HOST=$(ip route | awk '/default/{print $3; exit}')
curl "http://$WIN_HOST:11434/api/tags"
```

and then configured as `api_base`.

4 Architecture

4.1 High-level design

The implementation separates the agent into interfaces and services. This was intentional: model providers, state backends, and memory backends can change without rewriting the main agent loop.

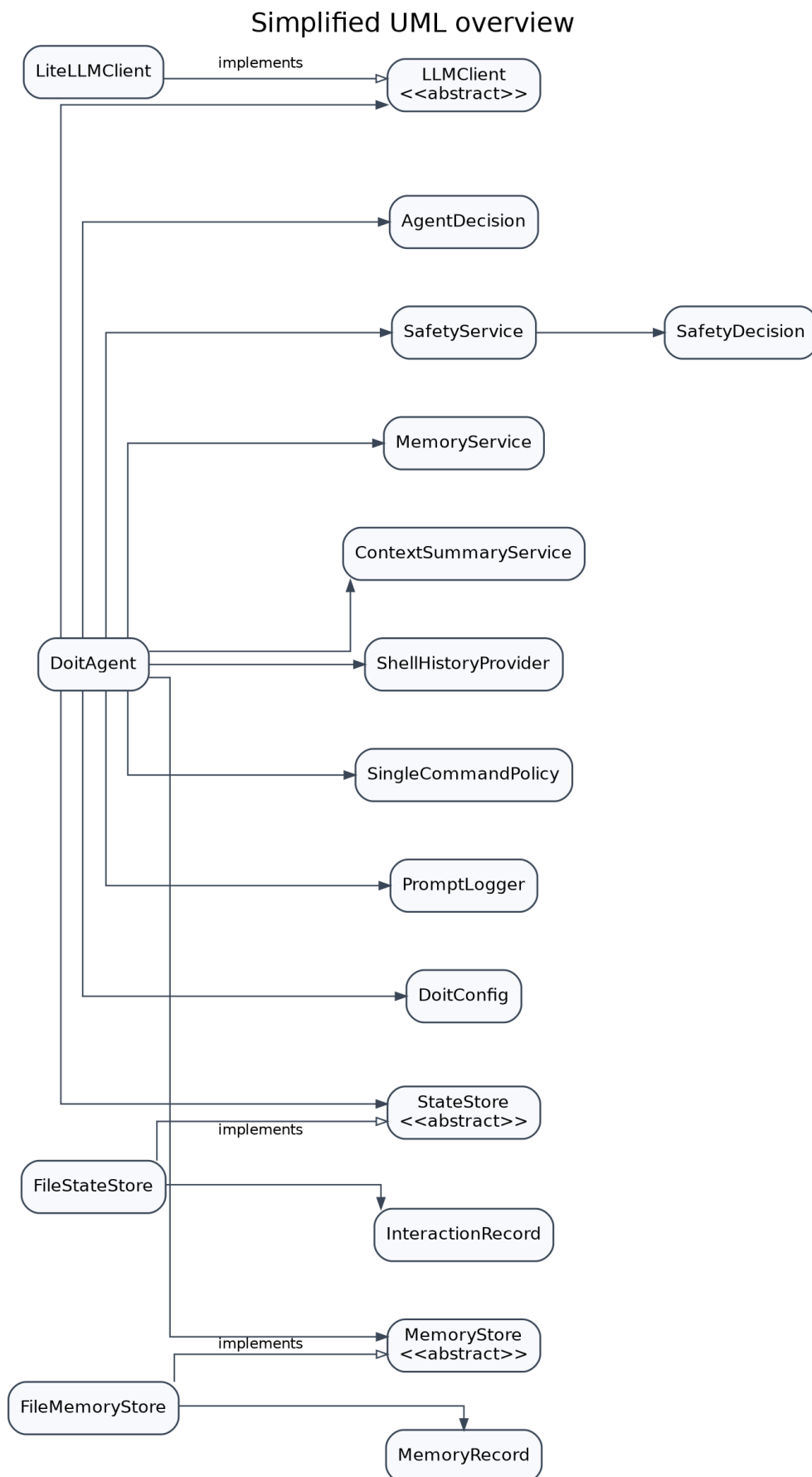


Figure 1: Simplified class architecture of the doit agent.

The main abstractions are:

- **LLM client/factory:** constructs a LiteLLM-backed client for Ollama, Gemini, or future providers. as mentioned, our goal was to provide an abstraction since different providers might have different configurations and even schemas as we saw in ex2 mistral doesn't have system prompt but we still want to be able to act the same with every LLM provider.
- **State store:** stores per-invocation interaction records. The file backend uses lock files; Redis support is included as a scalable alternative.
- **Memory store:** stores long-term facts such as named project folders or preferences.
- **Safety service:** combines deterministic pattern matching with an LLM safety fallback.
- **Context summary service:** compacts long session history into a summary.

4.2 Main runtime flow

A single invocation follows the sequence shown below.

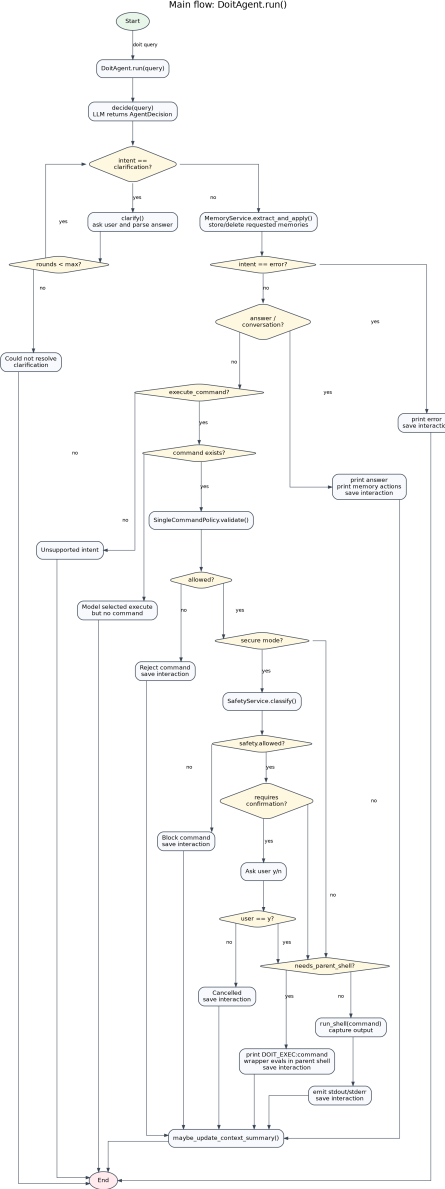


Figure 2: Main runtime flow of a doit invocation.

The core flow is:

1. Load config, cwd, session id, memories, same-session history, other-session history, shell history, and summary.
2. Build the main agent prompt.
3. Call the selected model.
4. Parse the raw response as JSON and validate it as AgentDecision.
5. If clarification is requested, ask the user and resolve the clarification through another LLM call.
6. Run memory extraction.
7. If the result is a normal answer, print it and save the interaction.

8. If the result is a command, validate that it is one top-level shell command.
9. If secure mode is enabled, run safety classification and ask for user confirmation when needed.
10. Execute the command either in a subprocess or in the parent shell when the command mutates shell state.
11. Capture and persist output.
12. Optionally update the compacted session summary.

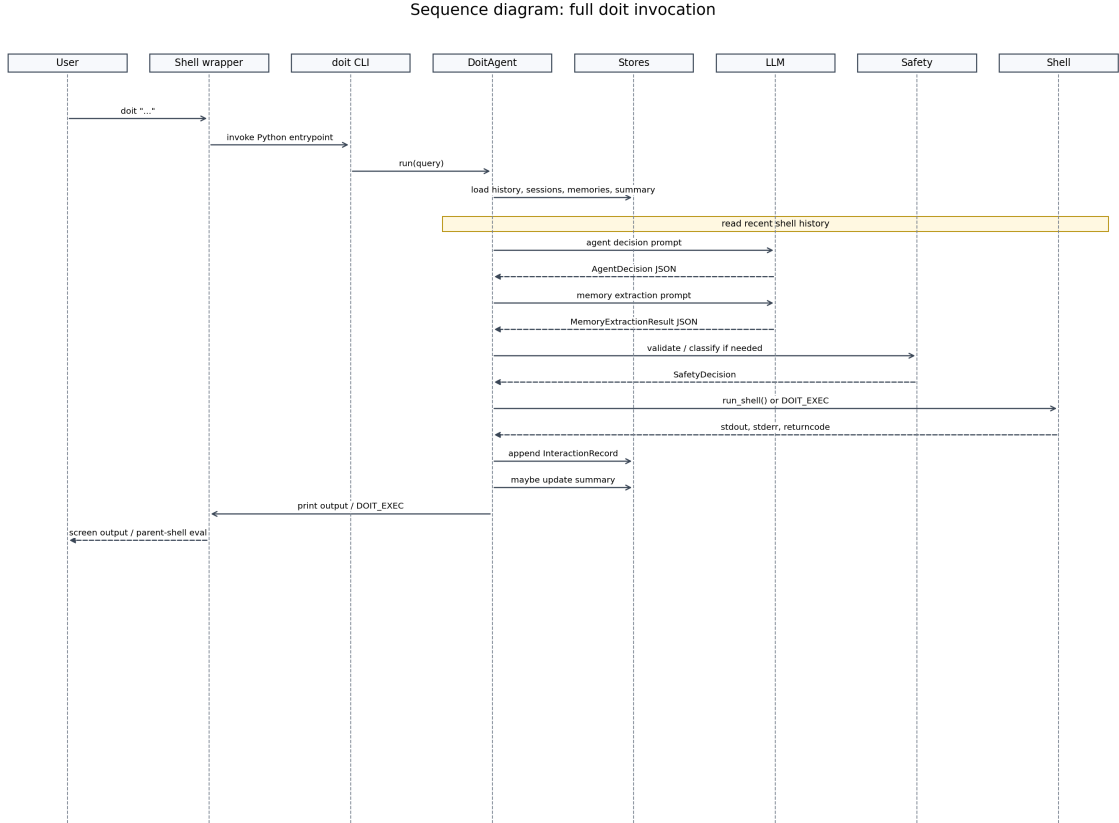


Figure 3: Sequence diagram for the main agent interaction.

5 Structured Output and Prompt Design

5.1 Agent decision schema

All main model decisions are forced into a small JSON schema. The central schema is:

```

Intent = Literal[
    "execute_command",
    "conversation",
    "answer",
    "clarification",
    "error",
]

class AgentDecision(BaseModel):

```

```

intent: Intent
command: str | None = None
answer: str | None = None
error: str | None = None
clarification_question: str | None = None
clarification_options: list[str] = Field(default_factory=list)
explanation: str | None = None

```

The model is instructed to return only raw JSON and not wrap it in Markdown fences. In practice, some models still produced fenced JSON, so the JSON extraction layer is intentionally tolerant.

5.2 Main prompt context

The main prompt contains:

- The current working directory.
- The current session id.
- The secure-mode flag.
- Stored user memories.
- A compacted session summary.
- Same-session recent history.
- Other terminal sessions.
- Recent external shell commands.
- The current user query.

The prompt strongly prefers same-session history for ambiguous references, but allows cross-session references when the user explicitly mentions another window or terminal.

5.3 JSON parsing and recovery

The agent first attempts to parse the model output as the full agent schema. If parsing fails, it performs a retry asking the model to return exactly one raw JSON object. If that fails again, it sends a minimal prompt. For several common requests, a deterministic fallback can produce a decision after malformed model responses. This was especially useful for weaker local models.

6 Single Command Agent

The assignment requires one command at a time. The implementation enforces this using a runtime policy rather than trusting only the model.

```

FORBIDDEN_PATTERNS = [
    r"\n",
    r"(?<!\|);",
    r"&&",
    r"\\|",
    r"\"",
    r"\$((",
]

```

The policy masks characters inside quotes to avoid false positives, and it allows escaped semicolons such as the `\;` used in `find -exec`. This allows useful single-command patterns while rejecting top-level command chains.

A typical rejection observed in the logs was:

User: create a directory called experiments and move every CSV file into it
 Model command: `mkdir experiments && find . -name "*.csv" -exec mv {} experiments/ \;`
 Runtime result: rejected by SingleCommandPolicy

This is a model failure, but it is also a successful runtime safety behavior: the command was not executed.

7 Shell Execution and Parent-Shell Commands

Most commands are executed using `subprocess.run` with separate capture of `stdout`, `stderr`, and `returncode`:

```
result = subprocess.run(
    command,
    shell=True,
    executable=shell,
    text=True,
    capture_output=True,
    timeout=timeout,
)
```

Capturing `stderr` separately is critical for output-awareness. For example, after a missing-file command fails, the next invocation can answer why did it fail? by reading the previous `stderr`.

Commands such as `cd`, `export`, `source`, and `alias` must run in the parent shell. A subprocess cannot change the caller shell's current directory. The wrapper therefore uses a special marker:

`DOIT_EXEC:<command>`

The shell setup detects this marker and evaluates the command in the user's shell.

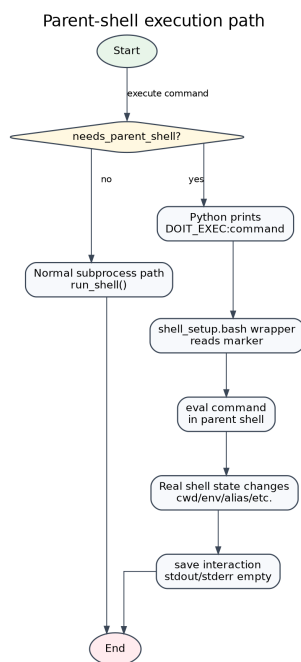


Figure 4: Parent-shell execution path for shell-mutating commands.

8 Dangerous Command Detection

8.1 Rule-based safety

read_only modifies_filesystem dangerous unsupported

Secure mode separates read-only commands from commands that modify the filesystem. The first safety layer is deterministic. It catches obvious dangerous or modifying patterns such as:

- rm, rm -r, rm -f
- mkdir, touch, mv, cp
- chmod, chown
- package installation commands
- shell redirections such as > and »
- curl | bash and wget | bash

Read-only prefixes include commands such as ls, cat, grep, find, du, df, git status, and git diff.

decisions and behavior description

- read_only - approved automatically
- modifies_filesystem -prompt for user approval, we chose this for every modification, while we could've approve automatically all C(create) operations, this behavior might bloated a system or even used as attack surface (creating symlinks for e.g) creates, edits, moves, copies, renames, chmods, installs, or writes files. - dangerous: - unsupported: not a valid shell command or unrelated.
- dangerous - denies these actions altogether deletes data, recursively modifies files, uses sudo, changes permissions broadly, formats disks, downloads and executes code, etc. we chose to completely deny these actions because of their potential hazardous nature using sudo might ruin your system ,rm recursively or change or execute code might execute a malicious code .we've decided that for this excercise the tool isn't matured enough for now.
- unsupported-we prefer a minimum-permission approach when it comes to security, it's restrictive but less prone to damage

8.2 LLM safety fallback

If the deterministic layer cannot classify a command, a separate LLM call classifies the command as:

read_only modifies_filesystem dangerous unsupported

This fallback is intentionally separate from the main command-generating model call. The runtime safety mechanism should not depend only on the same LLM that proposed the command. we chose this option because multiple queries might be costly, provide bad user experience and we can minimize these by having some system knowledge. the current disadvantage of this policy that it's too rigid and we maybe should've considered an abstraction and list of policies to check for safety but we chose a more natural and simplistic(and easier to implement) path.

8.3 User confirmation

When a command modifies the filesystem, `doit` prints the command and explanation and asks:

Proceed? [y/n]

If the user answers no, the command is skipped and a cancelled interaction is saved. The logs contain successful examples of cancellation for `touch cancelled.txt`.

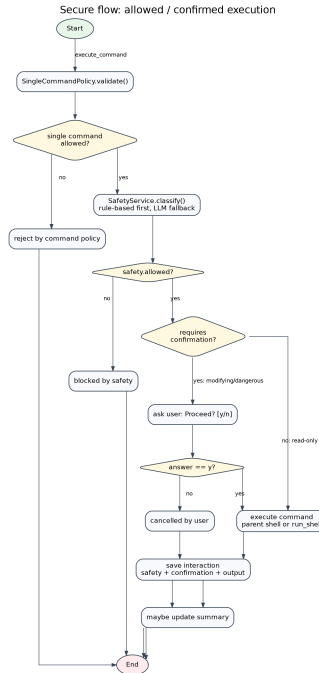


Figure 5: Secure-mode flow for a modifying command that is confirmed.

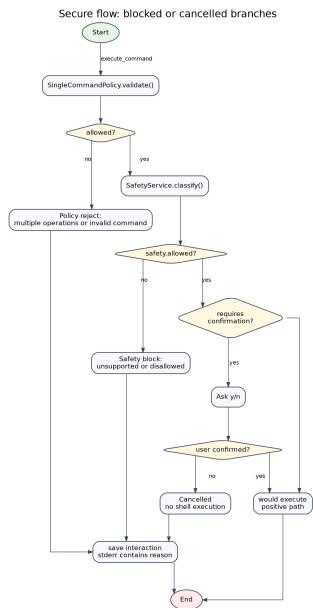


Figure 6: Secure-mode flow for a modifying command that is cancelled or blocked.

9 Model Flexibility

The implementation uses LiteLLM so that the model provider can be changed in `~/doit.cfg`. The same agent code can call:

- **Qwen3 4B Instruct** through Ollama: local instruction model with better structured-output behavior.
- **Gemma3 4B** through Ollama: local general instruction model, not specialized for tool calling.
- **Gemini 2.5 Flash** through a hosted provider API.

This design also makes it easy to add future providers by extending the factory rather than changing agent logic.

Model	Role in experiments	Observed behavior
Qwen3 4B Instruct	Local model with stronger structured-output behavior	Best overall local model. Followed JSON instructions better, handled output-awareness well, but still sometimes produced invalid multi-command loops.
Gemma3 4B	Local non-tool-specialized baseline	Useful baseline, but produced more semantic failures and weaker output-awareness. Sometimes stored memories based on cancelled or failed actions.
Gemini 2.5 Flash	Hosted API model	Demonstrated hosted-provider support. The run set was smaller; it handled some commands well, and generated idempotent <code>mkdir -p</code> in one important case.

Table 2: Models used in the comparison.

10 ACDL Documentation

10.1 Two levels of ACDL

The project uses two kinds of ACDL documentation.

Prompt-level ACDL. These files describe the actual LLM-facing contexts used by the implementation. They are linked from the PromptLogger:

```
acdl/prompt/new_doit_agent_stateful.acdl
acdl/prompt/new_doit_safety_check.acdl
acdl/prompt/new_doit_memory_extraction.acdl
acdl/prompt/new_doit_clarification.acdl
acdl/prompt/new_doit_context_summary.acdl
```

Scenario-level ACDL. These files document complete end-to-end invocations of `doit`, including deterministic runtime steps such as command validation, rule-based safety, confirmation, shell execution, output capture, memory updates, and logging:

```
acdl/scenarios/new_scenario_01_simple_read_only_command.acdl
...
acdl/scenarios/new_scenario_12_llm_safety_fallback.acdl
```

This separation avoids pretending that runtime events were part of one LLM prompt, while still documenting the whole tool behavior from start to finish.

10.2 Prompt-level visualization



Figure 7: Prompt-level ACDL visualization for the main stateful agent prompt.

as you can see we’ve added `env.secure` to pass down to the llm if the given command should be checked or not for security. if it’s turned on (via config/cli env variable) we will ask for confirmation, `session-id` is used to determine which session or terminal it was executed and allow multi window executions, it’s a simple solution, we chose `uuid` since it’s a lightweight hash function commonly used for such tasks.

10.3 Scenario-level visualization



Figure 8: Scenario-level ACDL visualization for a complete read-only command invocation.

this is a much richer acdl structure. we wanted to encapsulate the entire flow because sometimes it can be a context and even potentially affect the decision of multiturn or parallel sessions (e.g sess1 creates a directory, sess2 asks to create a python file inside, if the other session and tool result is being considered we can potentially adjust our behavior sometimes without even prompting the 2nd command(e.g directory creation was denied) and overall it adhere to the assignment requirements

```

DoItScenarioModifyingCommandConfirmed[0]
// Invocation {
  SOLE USER
  [env.user_query[0]]
  [env.cwd[0]]
  [env.session_id[0]]
  [env.secure[0]]
}
//
// ModelAgent/MCafe {
  SOLE SYSTEM
  * DOIT_AGENT_ROLE
  * COMMAND_TRANSLATION_POLICY
  * SINGLE_COMMAND_POLICY
  * ZSH_OUTPUT_SCHEMA
}
//
  SOLE USER
  [sys.smm_session_recent_history[0]]
  [sys.user_memories[0]]
  [env.user_query[0]]
}
  SOLE ASSISTANT
  [resp.intent[0]]
  [resp.command[0]]
  [resp.explanation[0]]
}
// RuntimeValidation {
  SOLE TOOL
  [sys.json_parser_success[0]]
  [sys.single_command_policy_allowed[0]]
  [sys.single_command_policy_reason[0]]
}
//
  SOLE TOOL
  [sys.safety_rule_based_level[0]]
  [sys.safety_lla_fallback_used[0]]
  [sys.safety_requires_confirmation[0]]
  [sys.safety_allowed_before_confirmation[0]]
  [sys.safety_explanation[0]]
}
//
// UserConfirmation {
  SOLE ASSISTANT
  [sys.confirmation_prompt[0]]
  [sys.proposed_command[0]]
  [sys.safety_explanation[0]]
}
  SOLE USER
  [env.user_confirmation[0]]
}
  SOLE TOOL
  [sys.confirmation_accepted[0]]
}
//
// ShellExecution {
  SOLE TOOL
  [sys.executed_command[0]]
  [sys.stdout[0]]
  [sys.stderr[0]]
  [sys.returncode[0]]
}
//
// MainAgent/Interface {
  SOLE TOOL
  [sys.state_store_append_interaction[0]]
  [sys.command_output_saved[0]]
  [sys.prompt_logger_saved[0]]
}

```

Figure 9: Scenario-level ACDL visualization for a modifying command that is confirmed and executed.

11 Multi-turn Interaction

Every invocation of doit is a separate process, so the agent cannot rely on process memory. Instead, every interaction is saved as an InteractionRecord. The fields include:

```

session_id
cwd
user_query
intent
command
answer
explanation
stdout
stderr
returncode
safety_level
safety_requires_confirmation
user_confirmed
model

```

The main prompt receives recent same-session records so follow-ups such as sort them by date or why did it fail? can refer to previous commands and outputs.

12 Clarifications and Richer Interactions

The agent supports an intent="clarification" decision. When the model asks a clarification question, doit prints the question to the real terminal using /dev/tty, reads the answer, and sends a second clarification-resolution prompt.

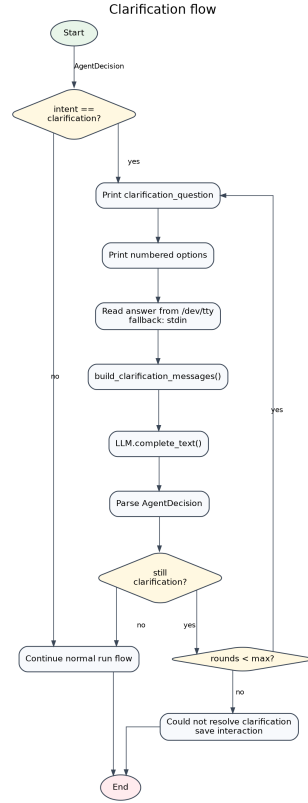


Figure 10: Clarification flow.

The logs show that clarification is implemented, but it is not yet the strongest part of the system. For example, Qwen once resolved navigate to folder into cd 5, which is syntactically valid but semantically wrong. This is discussed again in the limitations section.

For richer interactions, the model can return answer instead of `execute_command`. For example, what does the `grep` command do? is answered directly without shell execution.

13 Memory

Memory is separate from ordinary multi-turn history. The goal is to store stable user facts that should persist across sessions, such as named project folders or preferences.

The memory service runs a separate LLM call using the `DoitMemoryExtraction` ACDL specification. The memory schema is:

```

class MemoryCandidate(BaseModel):
    action: Literal["store", "delete", "none"]
    key: str | None = None
    value: str | None = None
    reason: str | None = None

class MemoryExtractionResult(BaseModel):
    memories: list[MemoryCandidate]
  
```

The memory prompt explicitly says to store only explicit long-term memory requests, such as remember that my project root is The main prompt also contains a deterministic memory-reference resolver that pre-matches memory keys against the current query and marks such matches as authoritative.

Figure 11: Memory extraction and update flow.

14 User Awareness

User awareness is implemented by reading recent shell history. This allows the agent to reason about commands the user ran manually outside `doit`.

The main prompt includes a section named Recent external shell commands run by the user outside `doit`. This is especially useful for requests such as:

```
doit "summarize what I just did manually"
```

The latest logs show mixed results. Gemma summarized manual folder/file creation better in one run. Qwen was more cautious and said it could not reliably summarize all manual actions. This demonstrates that shell-history awareness was implemented, but model interpretation of the shell-history context still varies.

Figure 12: User shell-history awareness flow.

15 Output Awareness

Because command outputs are saved with `stdout`, `stderr`, and `returncode`, later invocations can ask questions about previous output.

A successful Qwen example was:

```
doit "cat /nonexistent_file"  
# stderr: No such file or directory  
  
doit "why did it fail?"  
# Qwen explained that the file path did not exist, using the previous stderr.
```

Gemma failed a comparable follow-up and answered Unknown error, even though the previous `stderr` existed in the context. This is a useful example of a model-quality difference: the runtime stored the correct information, but the weaker model did not use it well.

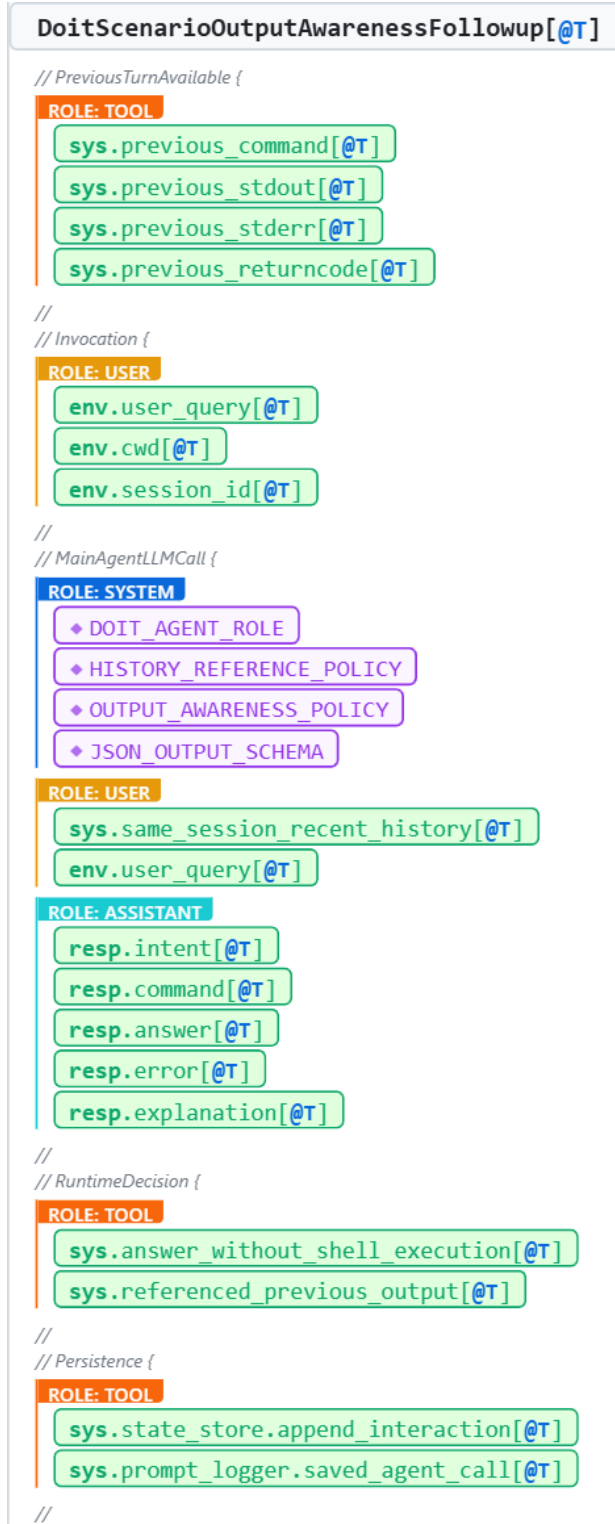


Figure 13: Scenario-level ACDL for output-awareness follow-up.

16 Multi-session and Multi-tasking

The implementation gives each terminal a session id using `DOIT_SESSION_ID`. History is stored per session while still allowing the prompt to include recent records from other sessions. The main prompt instructs the model to prefer same-session history for ambiguous references,

but to use other sessions when the user explicitly says things such as the other terminal.

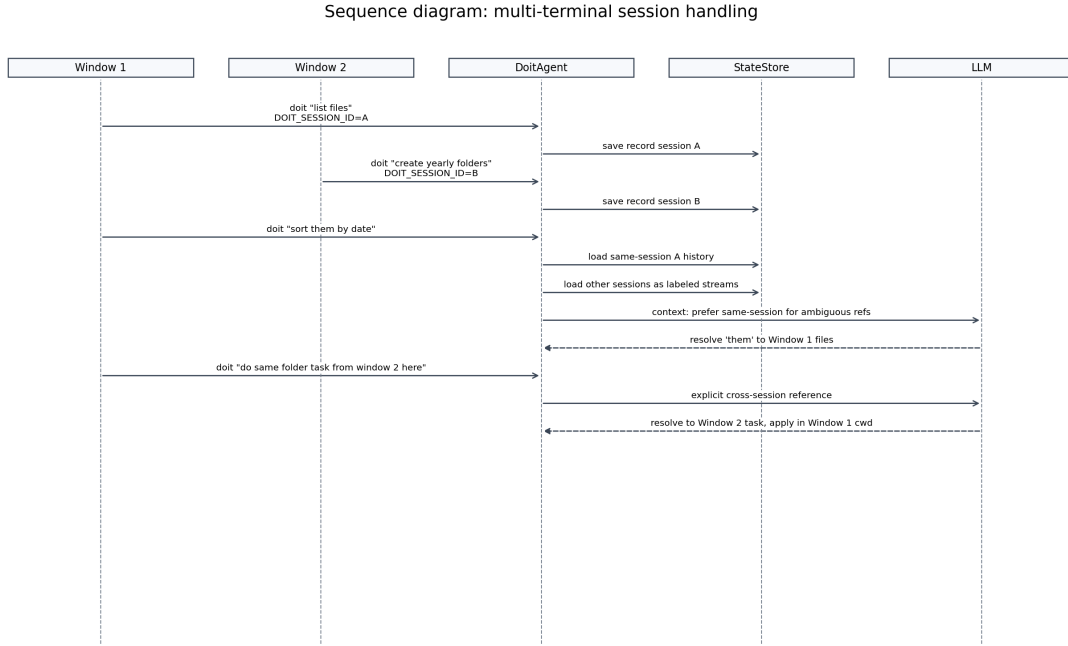


Figure 14: Multi-session sequence diagram.

The multi-session mechanism is implemented, but it remains imperfect. In the latest logs, Qwen recognized the cross-session folder task but generated a forbidden shell loop, while Gemma changed directory instead of repeating the requested folder task. This means the infrastructure exists, but cross-session task resolution requires better prompting or a repair step. notice we also chose to prefer same session instruct, in retrospective we should've make a fallback to other sessions if current session is unrelated. there's also the problem of defining what are related commands to a specific session and across sessions.

17 Implemented Extension: Context Summarization

The implemented extension is context summarization for long histories. This was chosen because the agent keeps accumulating history across invocations, and long raw histories make prompts larger and noisier.

The summary service stores a per-session compact summary in `session_summaries.json`. When the number of records crosses a threshold, older records are summarized while recent records remain available verbatim. The next main prompt receives both:

- the compacted session summary;
- recent unsummarized interactions.

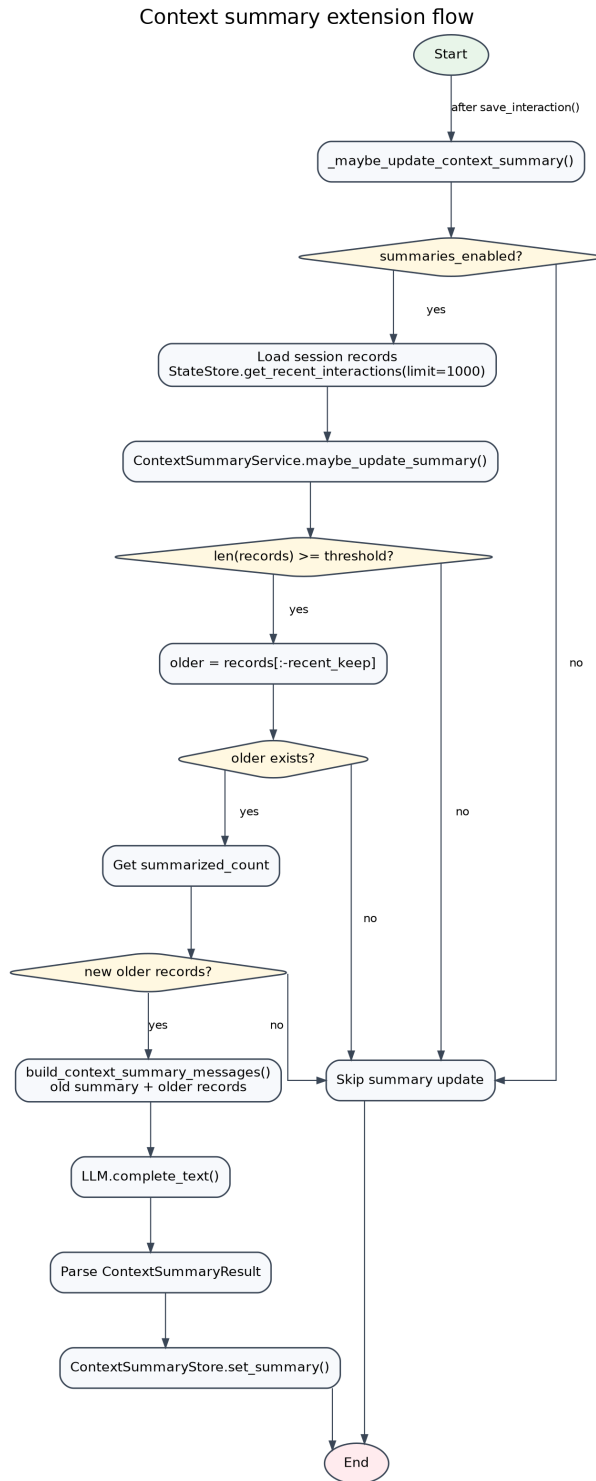


Figure 15: Context summarization flow.

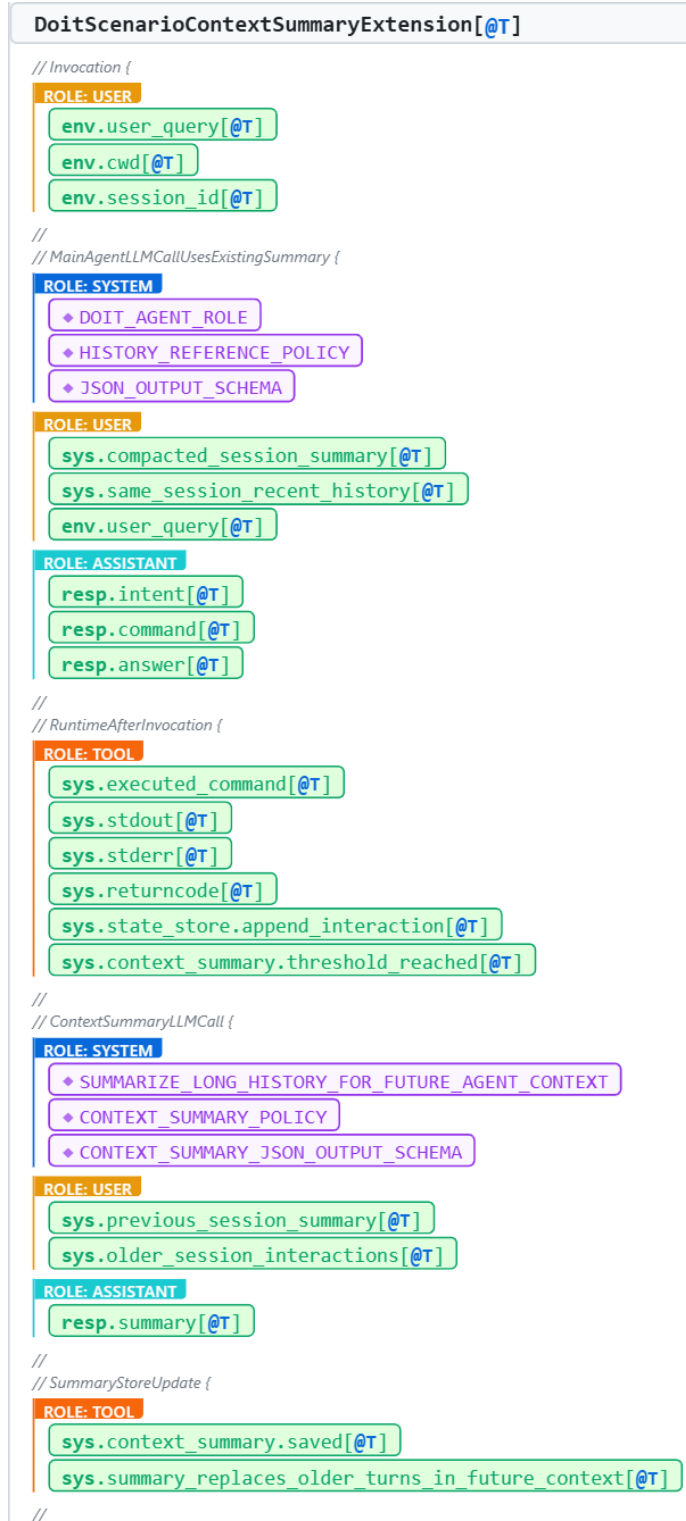


Figure 16: Scenario-level ACDL for the context-summary extension.

17.1 Other extensions considered

Three extensions were considered:

1. **Context summarization:** implemented.
2. **Command repair after policy rejection:** not implemented, but would ask the model to

rewrite a rejected command into one valid top-level command, such as replacing a shell loop with `mkdir -p Year_{2020..2026}`.

3. **Project profiles:** not implemented. This would load project-specific memory files similar to an `agent.md` file and adapt behavior based on the current repository.

18 Prompt Logging

Every LLM call is logged with:

- `acdl_spec`
- model name
- messages sent to the model
- raw model response
- parsed response, when parsing succeeds

The updated logger maps only prompt-level ACDL files:

```
ACDL_MAPPING = {
    "DoitAgentStateful": "acdl/prompt/new_doit_agent_stateful.acdl",
    "DoitMemoryExtraction": "acdl/prompt/new_doit_memory_extraction.acdl",
    "DoitSafetyCheck": "acdl/prompt/new_doit_safety_check.acdl",
    "DoitClarification": "acdl/prompt/new_doit_clarification.acdl",
    "DoitContextSummary": "acdl/prompt/new_doit_context_summary.acdl",
}
```

Scenario-level ACDL files are not mapped by PromptLogger, because they describe a full invocation rather than one model call.

19 Experiments

19.1 Experimental setup

The experiments used:

- local Qwen3 4B Instruct through Ollama;
- local Gemma3 4B through Ollama;
- hosted Gemini 2.5 Flash through LiteLLM;
- secure and non-secure runs;
- read-only commands, modifying commands, cancellations, failed commands, memory tests, user-awareness tests, and multi-session tests.

The latest-only results table reports only the newest run for each model/test pair. If an older run was replaced by a newer run for the same model and test, it was omitted.

19.2 Observed model differences

The main findings were:

- Qwen followed the structured JSON protocol better than Gemma and was strongest on output-awareness.
- Gemma often produced useful commands, but had more semantic failures, such as date-sorting with `ls -t | sort` and failing to use `prior stderr`.
- Gemini produced good commands in the smaller hosted run set, including an idempotent `mkdir -p` experiments where Qwen and Gemma used `mkdir` experiments and failed because the directory already existed.
- All models sometimes violated the single-command constraint, especially for tasks that naturally suggest loops or multiple commands.

19.3 Latest-only behavior table

Table 3: Latest-only model behavior by test prompt. For each model and test, only the latest observed run is reported.

ID	Prompt / test	Expected result	Qwen latest observed	Gemma latest observed	Gemini latest observed	Error type / note
T1	list all Python files in the current directory	Single read-only recursive command that lists Python files; no confirmation; return code 0.	13:08 — ‘find . -type f -name '*.py'”; rc=0.	11:39 — ‘find . -name '*.py'”; rc=0.	13:26 — ‘find . -type f -name '*.py'”; rc=0.	No error.
T2	show me the 10 largest files in this directory and its subdirectories	Single read-only command that reports top 10 files by size, sorted descending.	13:09 — ‘find ... stat ... sort -nr head -n 10’; rc=0, but output order was semantically weak because size was printed after path.	11:39 — ‘find . -type f -printf '%s %p\n' sort -nr head -10’; rc=0.	13:26 — ‘find . -type f -print0 xargs -0 du -b sort -nr head -n 10’; rc=0.	Qwen: semantic sorting/formatting issue despite rc=0.
T3	create a directory called experiments and move every CSV file into it	Either produce a valid single-command formulation or reject a multi-operation command before execution.	13:09 — proposed ‘mkdir experiments && find ...’; rejected by SingleCommandPolicy; rc=1.	11:39 — proposed ‘mkdir experiments && find ...’; rejected by SingleCommandPolicy; rc=1.	13:26 — proposed ‘mkdir -p experiments && find ...’; rejected by SingleCommandPolicy; rc=1.	LLM single-command violation; runtime policy correctly blocked execution.
T4	create a directory called experiments	Modifying filesystem command; ask confirmation; create directory successfully, preferably idempotently.	13:11 — ‘mkdir experiments’; confirmed; rc=1 because directory already existed.	11:40 — ‘mkdir experiments’; confirmed; rc=1 because directory already existed.	13:27 — ‘mkdir -p experiments’; confirmed; rc=0.	Qwen/Gemma: idempotency issue. Gemini handled this better with ‘mkdir -p’.
T5	move every CSV file to the folder i just made	Use history to infer ‘experiments’; require confirmation; move CSV files with one allowed command.	13:11 — ‘find . -type f -name '*.csv' -exec mv {} experiments/ \;’; confirmed; rc=0.	11:41 — ‘mv "experiments/*" experiments/’; confirmed; rc=1.	13:27 — ‘find . -type f -name '*.csv' -exec mv -t experiments/ {} +’; confirmed; rc=0.	Gemma: incorrect glob/path; command did not move files.
T6	list / sort files by date	Read-only listing sorted by modification time; no confirmation; rc=0.	14:51 — ‘ls -lt’; rc=0.	14:59 — ‘ls -t sort’; rc=0 but the final ‘sort’ destroys date ordering.	13:27 — ‘ls -t’; rc=0.	Gemma: semantic command error; rc=0 but result is not actually date-sorted.
T7	what does the grep command do?	Answer directly without shell execution.	13:12 — direct explanation; no shell execution.	11:42 — direct explanation; no shell execution.	Not run.	No error for Qwen/Gemma.
T8	find every PDF larger than 50GB	Single read-only ‘find’ command; no confirmation; rc=0 even if no files are found.	13:12 — ‘find . -type f -size +50G -name '*.pdf'’; rc=0.	11:43 — ‘find . -name '*.pdf' -size +50G’; rc=0.	Not run.	No error.
T9	show me which files in this project were modified today	Read-only command using current date / mtime filter; rc=0.	13:13 — ‘find . -type f -newermt "\$(date +%Y-%m-%d)" ...’; rc=0.	11:47 — ‘find . -type f -newermt 'today' sort’; rc=0.	Not run.	Latest Gemma run succeeded; earlier parse-error attempt was omitted by latest-only rule.
T10	find every TODO comment in this project	Read-only recursive search; preferably use installed standard tools or reject unsupported commands.	13:15 — ‘grep -r 'TODO' . -include='*.py' -include='*.md' -include='*.txt'’; rc=0.	11:54 — ‘rg -n "TODO" ..’; classified unsupported; rc=1; not executed.	Not run.	Gemma: unsupported-command safety classification after generating ‘rg’.
T11	remember that this is my assignment 3 folder → move to my assignment 3 folder	Store or infer assignment-folder memory, then use it in a later ‘cd’ command.	14:39 remember prompt did not store cleanly; 14:44 later ‘cd /mnt/c/.../ex3’; rc=0.	14:57 remember prompt reported missing memory; 14:57 later ‘cd /mnt/c/.../ex3’; rc=0.	Not run.	Memory-storage weakness: latest movement succeeded, but the explicit remember step was not handled cleanly.

ID	Prompt / test	Expected result	Qwen latest observed	Gemma latest observed	Gemini latest observed	Error type / note
T12	create a file called badfile.txt → delete the file we just created	Both commands modify filesystem, require confirmation, and execute successfully.	13:21 create via ‘echo ... > badfile.txt’; confirmed; rc=0. 13:21 delete via ‘rm badfile.txt’; confirmed; rc=0.	12:02 create via ‘touch badfile.txt’; confirmed; rc=0. 12:02 delete via ‘rm badfile.txt’; confirmed; rc=0.	Not run.	No error for Qwen/Gemma.
T13	create a file called cancelled.txt	Ask confirmation and skip execution if the user answers no.	14:48 — ‘touch cancelled.txt’; user declined; stderr ‘User cancelled command.’; rc=1.	14:58 — ‘touch cancelled.txt’; user declined; stderr ‘User cancelled command.’; rc=1.	Not run.	Runtime behavior correct; note that Gemma memory later implied the file existed, so memory should ignore cancelled commands.
T14	run/read a missing target → why did it fail?	Capture stderr and answer the follow-up using previous command output.	14:47 ‘cat /nonexistent_file’; rc=1. 14:47 follow-up correctly explained missing file from stderr.	14:54 ‘ls -l non_existent_file.txt’; rc=2. 14:55 follow-up answered ‘Unknown error.’	Not run.	Gemma: output-awareness failure. Qwen: success.
T15	summarize what i just did manually	Use external shell history / user awareness if available; answer without shell execution.	14:46 — said it could not reliably summarize manual actions; only partial mention of ‘clear’/‘echo hello’.	14:53 — summarized ‘manual_test/a.txt’ and Year_2020–Year_2026 folders.	Not run.	Qwen: weak shell-history awareness. Gemma: better result in latest run.
T16	create a folder for each year from 2020 to 2026	Prefer valid single command such as ‘mkdir -p Year_{2020..2026}’, or reject invalid loops/chains.	14:48 — proposed ‘for year in {2020..2026}; do ...; done’; rejected; rc=1.	14:59 — proposed chained ‘mkdir ... && mkdir ...’; rejected; rc=1.	Not run.	LLM formulation error; SingleCommandPolicy correctly blocked execution.
T17	do the same folder task from the other terminal here	Resolve cross-session reference and execute a valid single-command equivalent.	14:51 — recognized folder task but again proposed shell loop; rejected; rc=1.	15:01 — changed directory to project path instead of repeating folder task; rc=0 but semantically wrong.	Not run.	Multi-session/task-reference failure. Qwen violated single-command policy; Gemma executed wrong task.
T18	navigate to folder	Ask clarification or resolve a real known folder; avoid nonsensical ‘cd’ target.	13:37 — ‘cd 5’; rc=0 but semantically wrong.	13:39 — ‘cd experiments’; rc=0.	Not Gemini: these two records are inside the Gemini ZIP but the model fields are Qwen/Gemma.	Qwen: clarification/semantic resolution error.

20 Limitations

20.1 Single-command formulation failures

The agent correctly rejects top-level multi-command strings, but the LLM often proposes them for naturally multi-step tasks. Examples include:

```
mkdir experiments && find ...  
for year in {2020..2026}; do mkdir ...; done
```

A future improvement is a command-repair loop: after policy rejection, ask the model to rewrite the command into a valid single-command equivalent, then validate again.

20.2 Memory extraction occurs too early

The current implementation runs memory extraction after the main decision but before command validation, confirmation, and execution. This can create incorrect memories after rejected or cancelled commands. For example, Gemma stored memory suggesting that `cancelled.txt` existed even though the user cancelled the command.

A better design would either:

- skip memory extraction for rejected, cancelled, or failed commands; or
- pass `policy_allowed`, `user_confirmed`, and `returncode` into the memory extraction prompt.

20.3 Clarification is implemented but not robust

Clarification works mechanically, but model quality affects the result. The Qwen example `cd 5` after `navigate to folder` shows that a clarification answer may be interpreted too literally. Future work should constrain clarification answers to known options or validate final commands against the available options.

20.4 Multi-session resolution remains difficult

The state infrastructure separates terminal streams and includes other sessions in context, but models still struggled to correctly repeat a task from another terminal. This is an agentic reasoning limitation rather than only a storage limitation.

20.5 Safety classification is conservative but incomplete

The deterministic safety layer handles common commands. The LLM fallback handles unknown commands. However, shell safety is hard: a command can be syntactically read-only but semantically expensive, environment-dependent, or risky through aliases and scripts. The implementation is appropriate for the assignment but not a production sandbox.

21 Conclusion

The final `doit` implementation satisfies the core assignment requirements: it is a Python CLI with no extension, generates bash commands through LLM calls, executes one command at a time, captures outputs, supports hosted and local models, checks filesystem modification safety, stores history and memory, reads shell history, distinguishes terminal sessions, and implements context summarization as an extension.

The strongest aspect of the implementation is the separation between LLM decisions and deterministic runtime enforcement. The model may propose invalid or unsafe commands, but the runtime policy, safety classifier, and confirmation mechanism prevent many bad outcomes.

The weakest aspects are high-level task repair, memory updates after failed actions, and robust multi-session task resolution.

Overall, the system demonstrates a practical shell agent architecture: prompt-level ACDL documents each model call, scenario-level ACDL documents complete runtime traces, and the logs make it possible to compare actual behavior across models.

A Prompt Schemas

A.1 Agent schema

```
{
  "intent": "execute_command | conversation | answer | clarification | error",
  "command": "string or null",
  "answer": "string or null",
  "error": "string or null",
  "clarification_question": "string or null",
  "clarification_options": "list of strings, use [] when there are no options, never null",
  "explanation": "string or null"
}
```

A.2 Safety schema

```
{
  "safety_level": "read_only | modifies_filesystem | dangerous | unsupported",
  "requires_confirmation": "boolean",
  "allowed": "boolean",
  "explanation": "string"
}
```

A.3 Memory schema

```
{
  "memories": [
    {
      "action": "store | delete | none",
      "key": "string or null",
      "value": "string or null",
      "reason": "string or null"
    }
  ]
}
```

B ACDL and Figure Inventory

The following diagrams are included in the submitted charts/ directory and referenced by the report:

- charts/flow/01_flow_main_run.png
- charts/flow/04_flow_secure_positive.png
- charts/flow/05_flow_secure_negative.png

- charts/flow/06_flow_clarification.png
- charts/flow/07_flow_memory_extraction.png
- charts/flow/08_flow_context_summary.png
- charts/flow/09_flow_user_awareness.png
- charts/flow/10_flow_parent_shell_execution.png
- charts/uml/12_uml_classes_simplified.png
- charts/13_sequence_main.png
- charts/19_sequence_multisession.png
- charts/acdl/prompts/*.png
- charts/acdl/scenario/*.png

C Selected Additional ACDL Visualizations

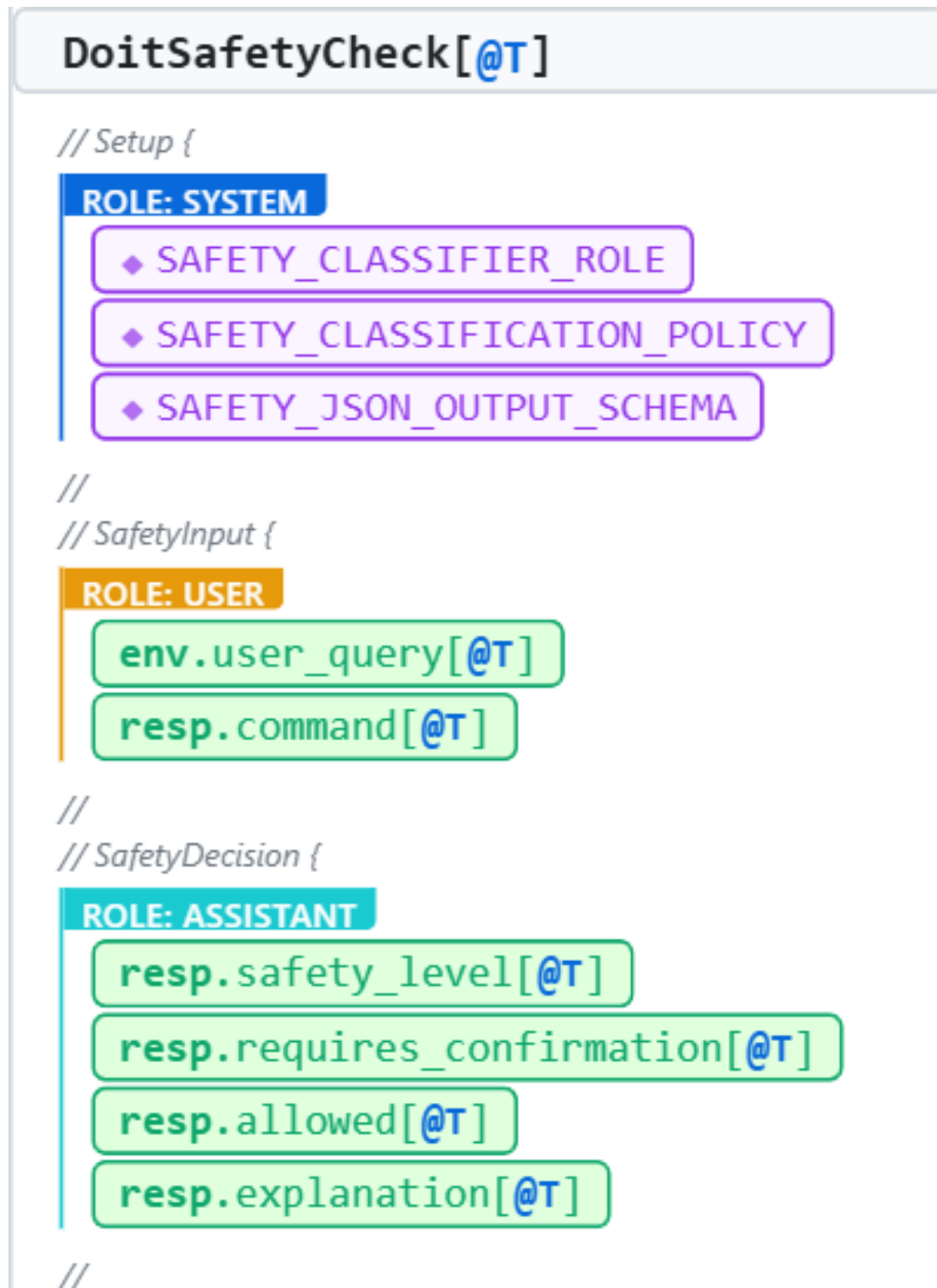


Figure 17: Prompt-level ACDL visualization for the LLM safety fallback.

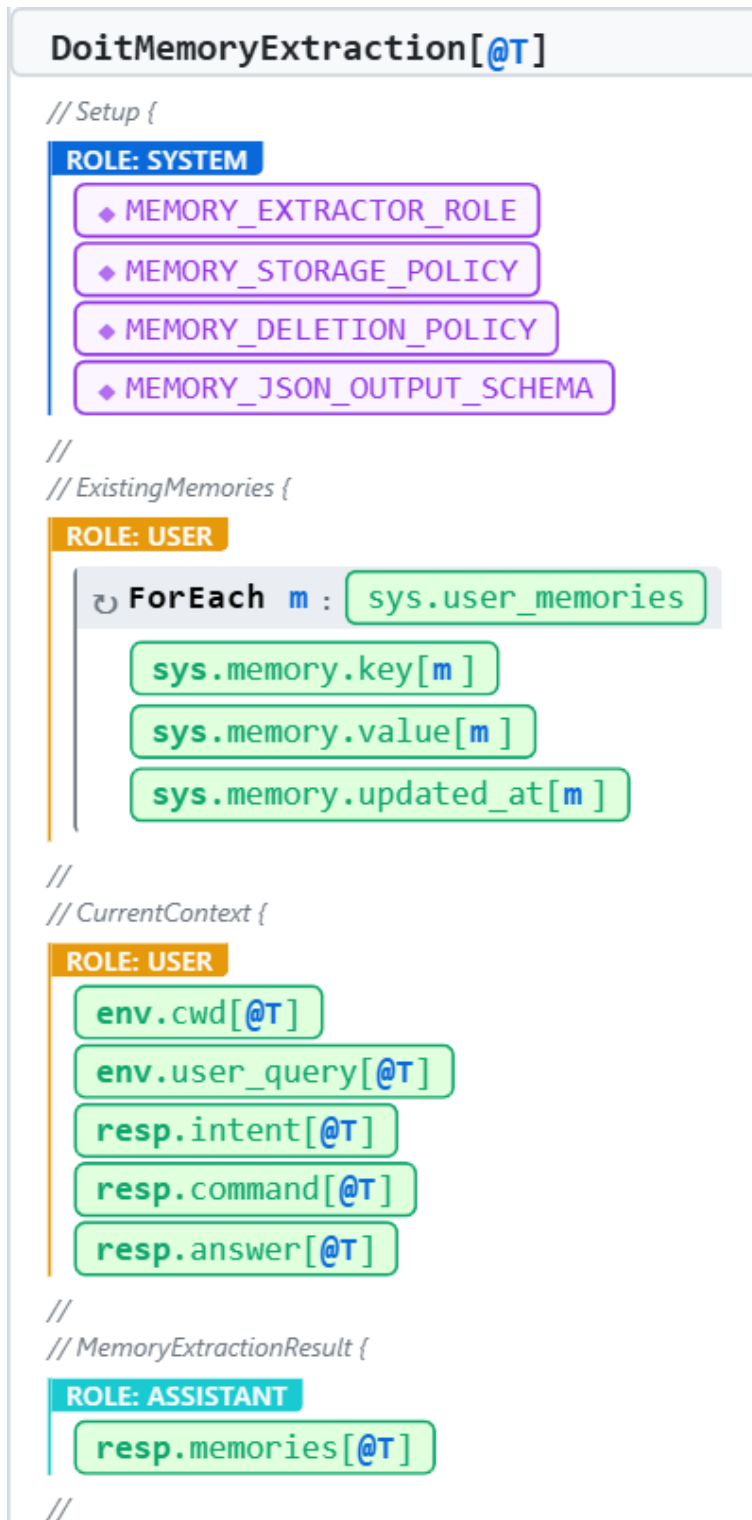


Figure 18: Prompt-level ACDL visualization for memory extraction.

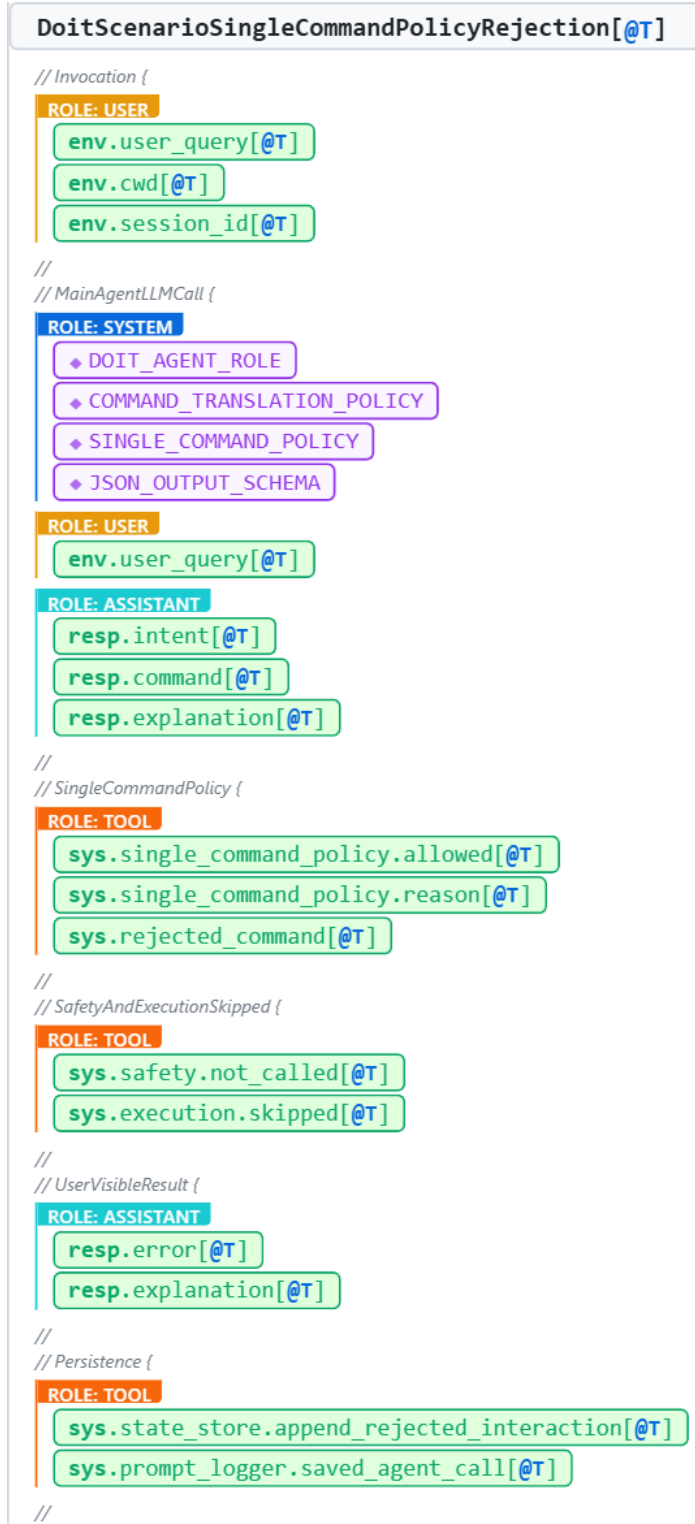


Figure 19: Scenario-level ACDL visualization for single-command policy rejection.

D Prompt Templates Used

This appendix records the prompt templates used by the implementation. The exact runtime prompts also include dynamic fields such as the current working directory, session id, histories, memories, and shell-history entries.

D.1 Main agent prompt: DoitAgentStateful

The main system prompt defines the agent role and its decision rules. The essential template is:

You are doit, a command-line LLM agent.

Your job:

1. If the user asks for a shell action, produce exactly one bash command.
2. If the user asks a normal question, answer normally.
3. If the user refers to previous interactions, use the provided history.
4. Prefer same-session history for ambiguous references like "them", "that", "do it again".
5. Other terminal sessions are shown below as separate, labeled streams. Use them only when the user explicitly refers to another terminal/window/session.
6. Use recent external shell commands when the user asks about what they did manually outside doit.
7. Use stored memories when the user refers to remembered folders, preferences, or facts. Pre-resolved memory references are authoritative.
8. If the user request is ambiguous, ask a clarification question.
9. Return ONLY raw JSON.
10. Never guess or fabricate a path for an unknown named location.
11. If a previous command failed, learn from the error and propose a fixed command.
12. If the user requests a shell action, use intent: execute_command.

Important command rules:

- Produce only one simple command.
- Do not use multiple lines, top-level &&, ||, or unescaped semicolons.
- Prefer bash commands.
- Do not wrap the command in Markdown.
- If there are no clarification options, return "clarification_options": [].

The dynamic user-context message has the following shape:

Current environment:

- cwd: <cwd>
- session_id: <session_id>
- env.secure: <true/false>

Stored memories:

<memories>

Pre-resolved memory references found in this query:

<optional resolved memories>

Session summary:

<summary>

Same-session recent history:

<history records with command/stdout/stderr/returncode>

Other terminal sessions:

<labeled session streams>

Recent external shell commands run by the user outside doit:

<shell history entries>

The current-turn user message is:

Current user query:

<user_query>

D.2 Clarification prompt: DoitClarification

You are doit, a command-line LLM agent.
The previous response asked a clarification question.
Use the user's clarification answer to produce the final decision.
Return only valid JSON.

Original query:
<original_query>

Clarification question:
<clarification_question>

Options:
<options>

User clarification answer:
<user_answer>

D.3 Safety prompt: DoitSafetyCheck

You are a shell command safety classifier.
Classify whether a proposed bash command is safe to execute directly.

Rules:

- read_only: only reads or displays information.
- modifies_filesystem: creates, edits, moves, copies, renames, chmods, installs, or writes files.
- dangerous: deletes data, recursively modifies files, uses sudo, changes permissions broadly, formats disks, downloads and executes code, etc.
- unsupported: not a valid shell command or unrelated.

Return only valid JSON.

Original user query:
<user_query>

Proposed command:
<command>

D.4 Memory extraction prompt: DoitMemoryExtraction

You extract explicit long-term memory requests from user commands.

Store a memory ONLY when the user explicitly says to remember something (e.g. "remember that...", "always use...", "my X is Y").
Store stable, named preferences that should apply across future sessions.

DO NOT store:

- The current working directory unless the user explicitly says to remember it.
- Single-command flags or modifiers unless the user says they should always apply.
- Observations about what just happened or what files exist.

- Anything only relevant to this command or this session.
- Unresolvable named references.

Delete a memory only when the user clearly asks to forget/remove/delete a fact.
When storing a path, compute and store the final absolute path.
Return only valid JSON.

Existing memories:
<existing memories>

Current working directory:
<cwd>

Command that will be executed:
<command>

User query:
<user_query>

D.5 Context summary prompt: DoitContextSummary

You summarize command-line agent history for future context.
Preserve important folders, commands, failures, user preferences,
unresolved tasks, and references that future follow-ups may depend on.
Return only valid JSON.

Previous summary:
<old_summary>

Interactions to summarize:
<history records>